System Design Notes: Reliability & Operations

Nov 5, 2026 · @Salike Hassan

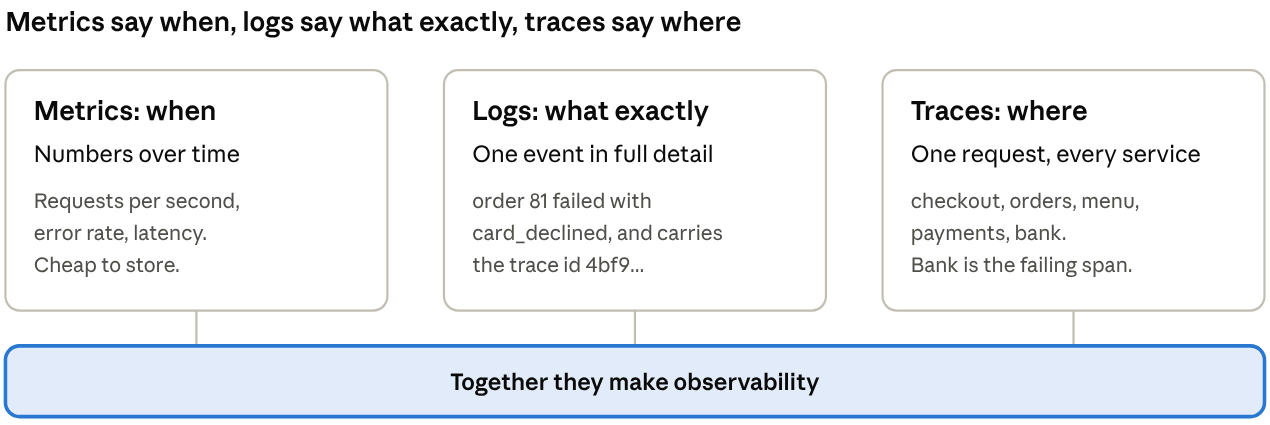
Observability for distributed systems

Observability matters in any modern app, distributed or a modular monolith. It is built from **three signals**: metrics, logs and traces. The order doesn't matter, what matters is that all three work together.

- **Metrics:** numbers over time, like requests per second, error rate, latency. Cheap to store, and great for spotting that something changed.
- **Logs:** the detailed record of one event, so we know exactly what happened.
- **Traces:** the path of one request across services, so we know where it happened.

Why metrics come first: with a good metrics setup, the moment latency or errors spike, the developer or stakeholder gets an alert on time. Without it, the app can be down for a long time and no one realizes. Logs and traces are what we open after the alert to find where and what.

Three signals at a glance



three signals · when, what exactly, where

The checkout example: metrics tell us the error rate went up, the trace shows the bank call failing, and the log says order 81 hit card_declined.

Metrics: golden signals, RED and USE

Google's SRE book names **four golden signals**. RED and USE are two shorthands built on them: **RED is for services, USE is for resources**.

Four golden signals (checkout service)

- **Latency:** how long a request takes. Checkout taking 4 seconds instead of 400 ms.
- **Traffic:** how much demand there is. Checkout requests per second.
- **Errors:** how many requests fail. Failed checkouts (5xx).
- **Saturation:** how full the service is. Threads, connection pool or queue close to the limit.

RED vs USE

RED (the service view): how many requests come in (rate), how many fail (errors), how long they take (duration). This is what the user feels.

USE (the resource view): one check for each CPU, disk, memory, connection pool.

- **Utilization:** how much of the resource is in use. CPU at 70% busy.
- **Saturation:** how much work is waiting because the resource is full. Requests queued for a DB connection.
- **Errors:** errors the resource itself reports. Disk read failures, dropped packets.

Utilization and saturation are not the same. A CPU at 100% with nothing waiting is busy but coping. Once a queue builds up, it is saturated and latency starts to climb.

How they work together: RED on checkout tells us users are hurting (duration is up). USE on its DB and CPU tells us why (the connection pool is saturated).

Percentiles: P90 and P99

P90 and P99 are the way we read **latency** (the Duration in RED). Sort all request times from fastest to slowest:

- **P90:** 90% of requests finished at or under this time. 1 in 10 was slower.
- **P99:** 99% of requests finished at or under this time. 1 in 100 was slower.
- **P50 (median):** the typical request.

Why not the average: the average hides the slow ones. Take 100 checkout requests: 98 take 100 ms and 2 take 5 seconds.

- Average is about 198 ms, which looks healthy.
- P90 is 100 ms.
- P99 is 5 seconds, so those 2 users are having a bad time.

The slow tail is also where the unhappy users, timeouts and retries live. So we alert on P99, not on the average.

How to derive it

1. Collect the duration of every request.
2. Sort them from fastest to slowest.
3. Take the value at position `p% x N`. With 1,000 requests, P99 is the 990th value.

We don't store every request in a metrics system. Instead the app records durations into **histogram buckets** (counts of requests under 50 ms, 100 ms, 250 ms...), and the metrics tool estimates the percentile from the buckets. In Prometheus: `histogram_quantile(0.99, ...)`.

If the durations are in a SQL Server table, we can calculate it directly:

```
SELECT DISTINCT PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY duration_ms)
OVER () AS p99_ms
FROM request_log;
```

Trap: don't average the P99 of each server to get the overall P99. Merge the histograms from all servers first, then calculate the percentile on the merged data.

Two metric traps

1. Averaging p99s

Five servers report a p99 of 120, 95, 3100, 110 and 105 ms. Averaging them gives about 706 ms, which matches nothing real: four servers are fine and one is very slow. **Average of p99s is not the system p99.**

- **Fix:** record **histograms**, which are counts of requests in latency buckets.
- Histograms from every server can be **added up** bucket by bucket. The percentile is then calculated on the merged histogram, later.

The average of p99s is not the system p99, merge histograms instead

p99 120 | p99 95 | **p99 3100** | p99 110 | p99 105

average of p99s is not the system p99



Histograms add up bucket by bucket, then the percentile is read from the merged one.

Bars are illustrative, not real counts.

p99 per server vs merged histograms

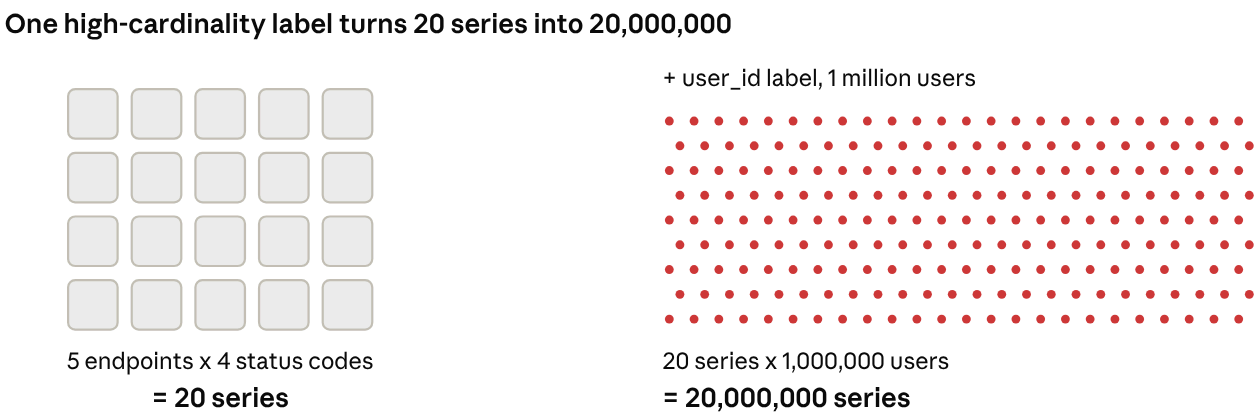
2. Cardinality

Cardinality is the number of distinct values a label can take.

A metric is a name plus labels, for example `http_requests_total{endpoint="/checkout", status="200"}`. Every **unique combination of label values is its own time series**, and each series is stored, indexed and billed separately.

- **Low cardinality:** endpoint (5 values), status code (4 values). Safe to use as labels.
- **High cardinality:** `user_id` (1 million values), `order_id`, `trace_id`, email. Almost every value is new, so each one creates a new series.

The number of series is the labels' cardinalities multiplied together: 5 x 4 = 20. Add `user_id` and it becomes 20 x 1,000,000 = **20,000,000 series**. That is how metrics bills explode.



Fix: keep labels low-cardinality. Per-user detail belongs in logs and traces.

cardinality · 20 series vs 20,000,000 series

- **20 million series is not 20 million requests.** It is 20 million separate time series to store and query, whatever the traffic is.
- **Sampling doesn't fix it.** Sampling is for traces and logs. For metrics the control is cardinality: choose labels carefully and set **cardinality limits**.
- **Fix:** keep metric labels low-cardinality (endpoint, status code). Per-user detail belongs in **logs and traces**, where each event is stored on its own and can carry `user_id`.

Structured logs

A plain text log like `payment failed for order 81 :(` is hard to search. A **structured log** writes the same event as fields (JSON), so we can filter on them:

```
{ "event": "payment_failed",
  "order_id": "81", "customer": "c_42",
  "error": "bank_timeout", "latency_ms": 3412,
  "bank": "acme", "trace_id": "4bf92f35..." }
```

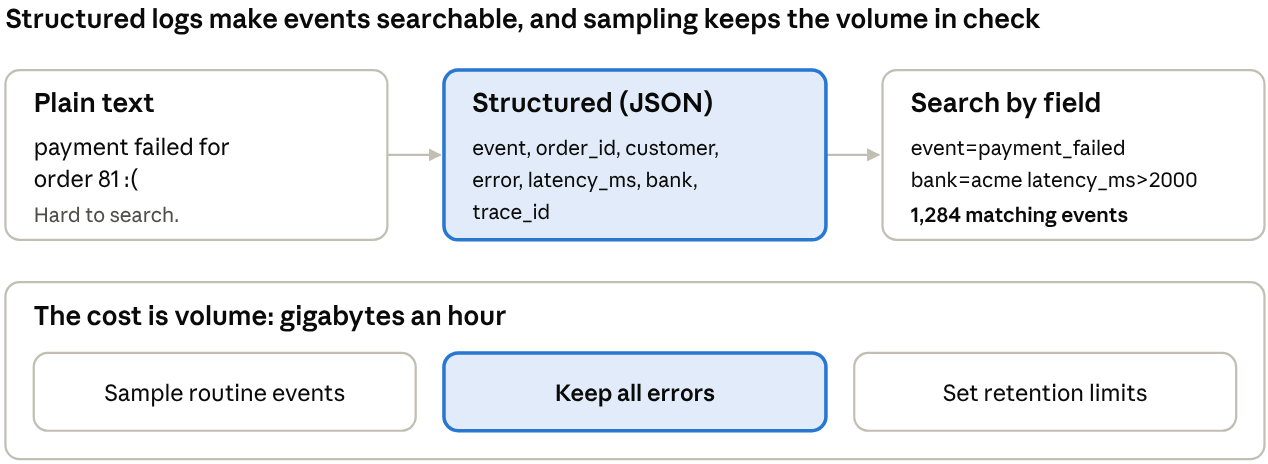
Now we can search: every failed payment in the last hour, for one bank, slower than 2 seconds.

```
event=payment_failed bank=acme latency_ms>2000 last=1h
-> 1,284 matching events
```

The `trace_id` field is what links a log line to its trace.

The cost is volume. A busy service can write gigabytes an hour. So teams:

- **Sample routine events:** keep only a share of the boring ones.
- **Keep all errors:** errors are stored in full.
- **Set retention limits:** logs are deleted after a fixed time.

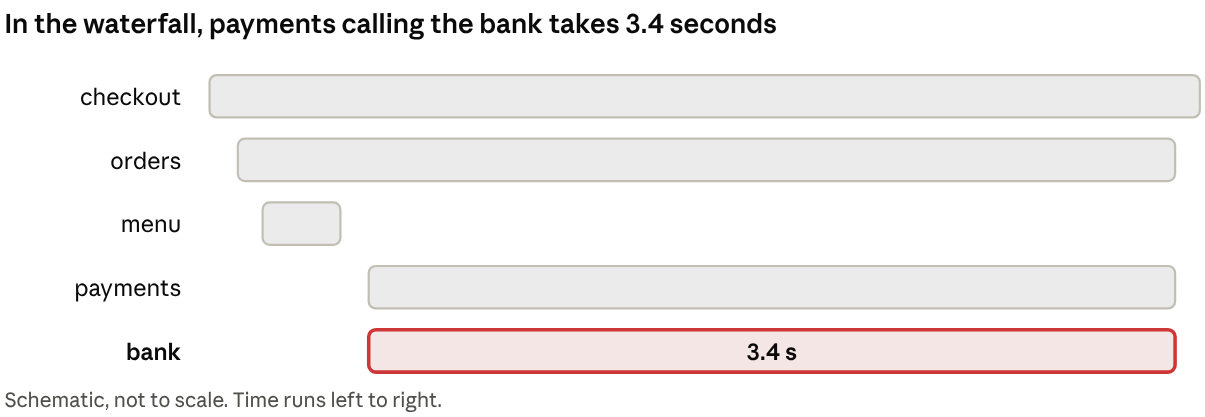


plain text vs structured logs · search and volume levers

Distributed tracing

When a checkout request enters the system, it gets a **trace id**. Every service that handles it records a **span**: what it did, when it started, and how long it took.

- The trace id is **propagated** from service to service with the request. That is how the spans from different services join into one trace.
- Laid out on a timeline, the tree of services becomes a **waterfall**, and the slow part jumps out.

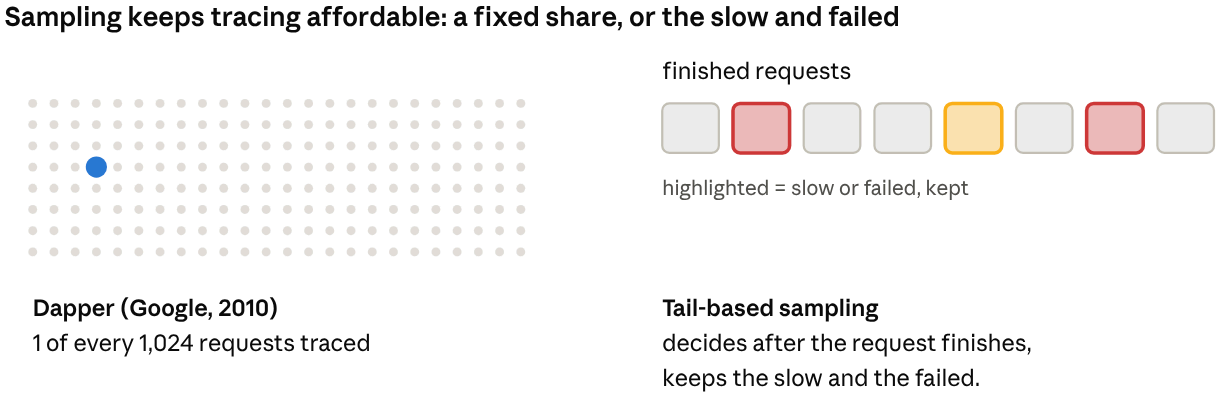


trace waterfall · checkout, orders, menu, payments, bank

Sampling

Tracing every request is costly, so we sample.

- **Dapper (Google, 2010)** made tracing practical at scale. The first version traced just 1 request in every 1,024, and that was still enough for most questions.
- **Tail-based sampling:** modern collectors decide **after the request finishes**, and keep all the slow and failed ones.



Dapper sampling vs tail-based sampling

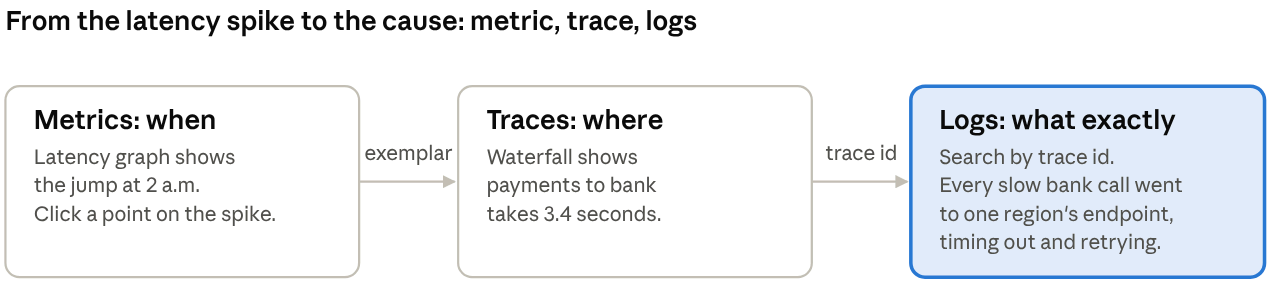
Why sampling is enough: most requests look alike, so a small share shows the normal path. The risk with a fixed share is missing the rare slow or failed request, which is exactly what we need during an incident. That is why tail-based sampling keeps all the slow and failed ones.

Trade-off: the collector has to hold all the spans of a request until it finishes before it can decide, so tail-based sampling costs more than a fixed share.

Sampling vs metrics: sampling is for traces and logs. Metrics are aggregates (counts and histograms) of every request, so they still show *when* something changed.

Solving the 2 a.m. page

This is where the three signals are linked together.



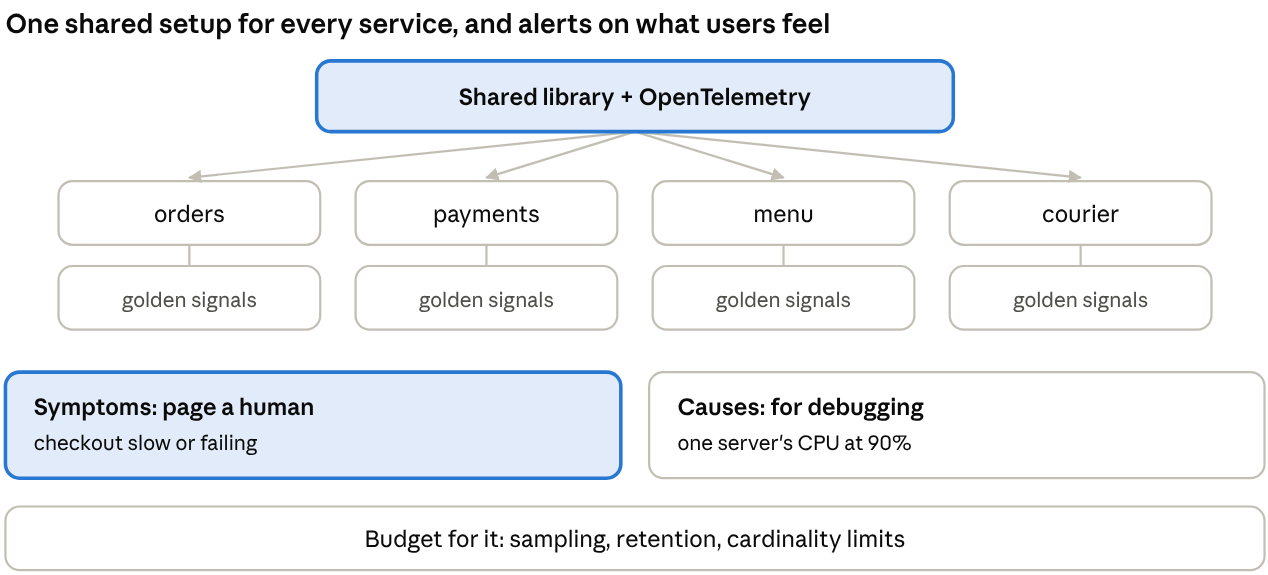
2 a.m. page · metric to trace to logs

1. **Metric:** the latency graph shows the jump. Click a point on the spike and it links to an example trace from that moment. That link is called an **exemplar**.
2. **Trace:** the waterfall shows payments calling the bank takes 3.4 seconds.
3. **Logs:** take the trace id and search the logs. Every slow bank call went to one region's endpoint, which is timing out and retrying.

```
trace=4bf9 bank=acme endpoint=eu-west-2 timeout retry=1
trace=4bf9 bank=acme endpoint=eu-west-2 timeout retry=2
trace=7c21 bank=acme endpoint=eu-west-2 timeout retry=1
```

Observability habits

- **One shared library plus OpenTelemetry:** every service (orders, payments, menu, courier) emits its signals the same way.
- **Basic golden-signals dashboard per service,** named the same way everywhere.
- **Alert on symptoms, not causes:** page a human for what users feel (checkout slow or failing), not for every cause (one server's CPU at 90%). Causes are for debugging, symptoms are for waking people up.
- **Budget for it:** sampling, retention and cardinality limits. Observability data can grow faster than the system it observes.



observability habits · shared setup, symptoms vs causes, budget

Recap

1. Metrics: when. Traces: where. Logs: what.
2. Golden signals, histograms, low-cardinality metrics.
3. Propagate trace ids, keep the slow and failed traces.
4. Link the signals, alert on what users feel.

Demo: Azure Application Insights and Datadog

The goal is to reproduce the 2 a.m. page with a small app, and show metrics, traces and logs linked together in either tool.

Demo app (same for both tools)

- Three small ASP.NET Core APIs: **checkout, orders, payments**. Payments calls a fake **bank** endpoint.
- A switch on the bank stub (an environment variable) that adds a delay and a failure rate, for example `BANK_DELAY_MS=3400`. That makes payments to bank the slow span, like the waterfall.
- Instrument with **OpenTelemetry**, so the same app can point at either tool.
- Write **structured JSON logs** with fields (`event`, `order_id`, `bank`, `latency_ms`, `error`), and keep the trace id on every log line.

- A traffic loop (a simple script or k6): normal traffic for a few minutes, then flip the bank switch.

Option 1: Azure Application Insights

1. Create an Application Insights resource and copy its connection string. Set it as the environment variable `APPLICATIONINSIGHTS_CONNECTION_STRING`.
2. Add the package and one line of startup code. Microsoft's current docs recommend the Microsoft OpenTelemetry Distro for new apps. The earlier `Azure.Monitor.OpenTelemetry.AspNetCore` package with `UseAzureMonitor()` still works.
3. Name each service with `OTEL_SERVICE_NAME` (the Cloud Role Name), so checkout, orders and payments show up as separate nodes in the Application Map.

```
// dotnet add package Microsoft.OpenTelemetry
using Microsoft.OpenTelemetry;

var builder = WebApplication.CreateBuilder(args);

builder.UseMicrosoftOpenTelemetry(options =>
{
    options.Exporters = ExportTarget.AzureMonitor;

    // Demo: keep every trace so nothing is missing
    options.AzureMonitor.TracesPerSecond = null;
    options.AzureMonitor.SamplingRatio = 1.0F;
});
```

4. **Sampling demo:** the distro samples at about 5 traces per second by default. Drop `SamplingRatio` to `0.1F` (about 10% of traces) and show traces going missing. Metrics are never sampled, so alert on metrics.
5. **Find the signals in the portal:** metrics on the Overview and Metrics charts, traces in the Application Map and the end-to-end transaction view (the waterfall), logs in the Logs blade with KQL.
6. **Alert on the symptom:** an alert rule on failed requests or checkout duration, not on CPU.

KQL for the P90 and P99 of the bank call:

```
dependencies
| where timestamp > ago(1h) and name has "bank"
| summarize p90 = percentile(duration, 90), p99 = percentile(duration, 99) by
bin(timestamp, 5m)
| render timechart
```

KQL for the logs of one trace (`operation_Id` is the trace id):

```
traces
| where operation_Id == "<trace id from the transaction view>"
| project timestamp, message, customDimensions
```

Option 2: Datadog

1. Install the Datadog Agent. On Azure App Service, use the Datadog extension for Azure App Service, which collects traces and custom metrics for .NET apps.
2. Add the Datadog .NET tracer (automatic instrumentation) and set `DD_ENV`, `DD_SERVICE` and `DD_VERSION`.
3. **Logs:** send them as JSON. Trace correlation is on by default from tracer version 3.24.0 (older versions need `DD_LOGS_INJECTION=true`), and the logs get `dd_trace_id` and `dd_span_id`. With Microsoft.Extensions.Logging, turn on log scopes and the JSON formatter.
4. **Find the signals:** a dashboard and a monitor for checkout latency and errors, the APM trace view for the waterfall, and the Log Explorer with a search like `service:payments @latency_ms:>2000`.
5. **Sampling demo:** traces and logs are sampled independently, so a log can carry a trace id whose trace was not kept. That is not a misconfiguration, it is the sampling lesson.
6. **Cardinality demo:** Datadog bills custom metrics by unique tag combinations, so adding a `user_id` tag to a metric is the live version of the 20,000,000 series example.

Where each signal lives

Signal	Application Insights	Datadog
Metrics (when)	Overview and Metrics charts, alert rules	Dashboards and monitors

Signal	Application Insights	Datadog
Traces (where)	Application Map, end-to-end transaction view	APM trace view
Logs (what exactly)	Logs blade (KQL), `operation_Id` = trace id	Log Explorer, logs carry `dd_trace_id`
Sampling	Rate-limited by default, or fixed percentage. Metrics never sampled	Ingestion controls. Traces and logs sampled independently

Demo flow

1. Run normal traffic: the golden-signals dashboard is flat and P99 is healthy.
2. Flip the bank switch: the latency graph jumps and the alert on the symptom fires (**metrics: when**).
3. Open a sample trace from the slow requests: the waterfall shows payments to bank at about 3.4 seconds (**traces: where**).
4. Search the logs by that trace id: timeouts and retries on one endpoint (**logs: what exactly**).
5. Show the cost levers: lower the sampling ratio, then add a `user_id` metric label and watch the series count.

Sources

- Azure Monitor OpenTelemetry: enable (Microsoft Learn source)
- Configure OpenTelemetry in Application Insights: sampling, Cloud Role Name
- Datadog: correlating .NET logs and traces
- Datadog: correlate logs and traces